

```
$ psql -c "SELECT ... FROM students JOIN courses ON ..."
```

SQL JOIN

// 0000000000000000

INNER

□□

LEFT

□□□□

RIGHT

□□□□

FULL

□□

CROSS

□□□□

\$ cat query-plan.md



01 /  **JOIN**

□□□□□□□□□□□□□□

02 /  

□□□□□□□

03 /   

ON vs WHERE □□□

04 /  

□□□□□□□□□□

□□□□□□□□□□□□□□□□□□□□□□□□ = LEFT□□□ = INNER□

PART 01

NO GLYPH | NO GLYPH | JOIN

🐼 ID: 1,2,3,4 × 🐼 ID: 2,3,5

INNER JOIN



```
1 SELECT
2     s.student_id,
3     s.name AS student_name,
4     c.course_name
5 FROM students AS s
6 INNER JOIN courses AS c
7     ON s.student_id = c.student_id;
```



RESULT SET

000 ID01, 2, 3, 4

000 ID02, 3, 5

→ 0000 ID02, 3

x 00000001, 40000

x 0000000500000

LEFT JOIN



```
1  SELECT
2      s.name,
3      c.course_name,
4      CASE
5          WHEN c.course_name IS NULL
6          THEN 'NULL' ELSE 'NULL'
7      END AS status
8  FROM students AS s
9  LEFT JOIN courses AS c
10     ON s.student_id = c.student_id;
```

RESULT SET

→ ID 1, 2, 3, 4

→ 1 4 NULL



LEFT JOIN □□□□□

NOT RUN NOT RUN 1 NOT RUN NOT RUN NOT RUN NOT RUN NOT RUN NOT RUN NOT RUN NOT RUN

```
1 SELECT c.customer_name
2 FROM customers c
3 LEFT JOIN orders o
4     ON c.customer_id = o.customer_id
5 WHERE o.order_id IS NULL;
6 -- □□□□□□
```

NOT RUN NOT RUN 2 NOT RUN NOT RUN NOT RUN NOT RUN NOT RUN NOT RUN NOT RUN NOT RUN

```
1 SELECT s.name,
2         COUNT(c.course_id) AS course_count
3 FROM students s
4 LEFT JOIN courses c
5     ON s.student_id = c.student_id
6 GROUP BY s.student_id, s.name;
7 -- □□□□□□ count = 0
```

LEFT JOIN + IS NULL □□□□□□□□□□—INNER JOIN □□□

RIGHT FULL OUTER CROSS

RIGHT JOIN

□□□□□□□□□□

A LEFT JOIN B $\equiv$ B RIGHT JOIN

A □  LEFT □□□

FULL OUTER JOIN

 NULL□

MySQL  LEFT $\cup$ RIGHT

□ UNION □□□

CROSS JOIN

 3 $\times$ 4  = 12 

 $\times$ 

```
1  -- MySQL 实现 FULL OUTER JOIN
2  SELECT * FROM students s LEFT JOIN courses c ON s.student_id = c.student_id
3  UNION
4  SELECT * FROM students s RIGHT JOIN courses c ON s.student_id = c.student_id;
```

PART 02

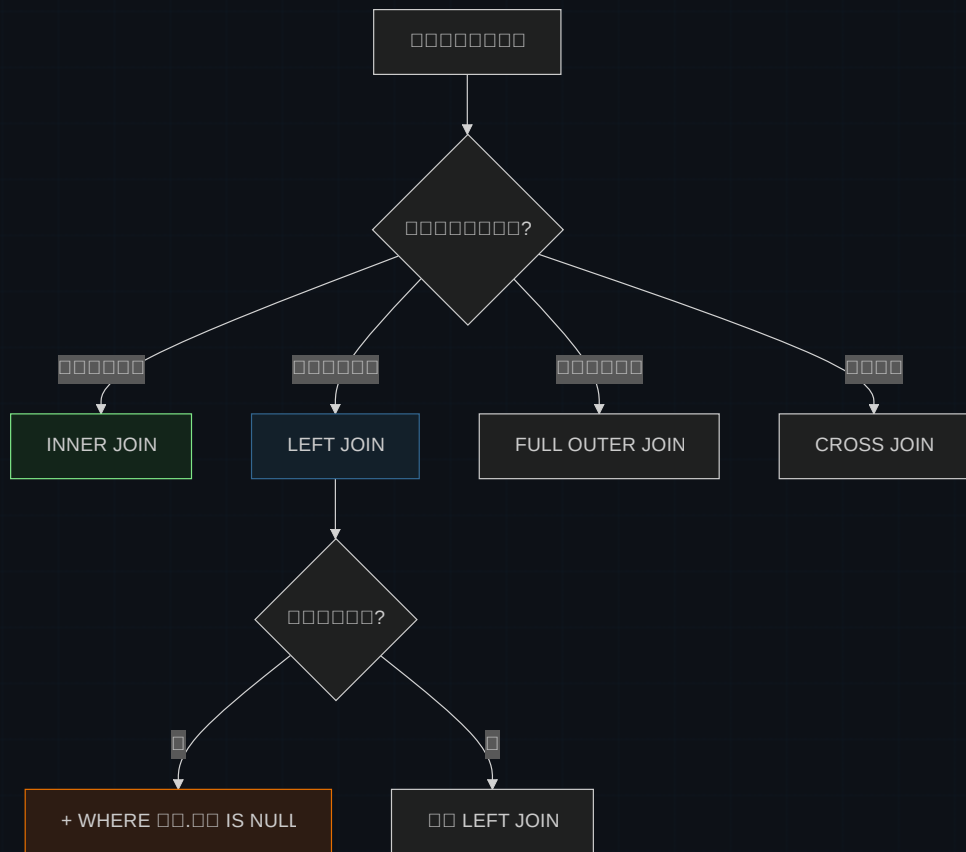
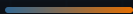


$$\square\square\square + \square\square\square\square$$

JOIN

JOIN	Table 1	Table 2	Result	Performance
INNER JOIN	10000	10000	1000	★★★★★
LEFT JOIN	100	10000	10000 NULL	★★★★★
RIGHT JOIN	10000	100	100 NULL	★★
FULL OUTER	100	100	10000 NULL	★★
CROSS JOIN	100	100	1000000	★

\$ 90% 1000000 INNER 100 LEFT — 100000000



PART 03

□□□□□ON vs WHERE

90% ⚡ LEFT JOIN bug □□□

CHECKPOINT_01

LEFT JOIN courses c ... WHERE c.course_name = 'Math' □□□□

A □□□□□□□□□□ Math □□□ NULL

B □□□□ Math □□□—LEFT JOIN □ WHERE □□ INNER □□

C □□□□□LEFT JOIN □□□□□ WHERE



```
1  -- ❌ WHERE JOIN 过滤 → NULL 数据 INNER 连接
2  SELECT * FROM students s
3  LEFT JOIN courses c ON s.student_id = c.student_id
4  WHERE c.course_name = 'Math'; -- 过滤 Math 课程
5
6  -- ✅ ON JOIN 过滤 → 数据保留
7  SELECT * FROM students s
8  LEFT JOIN courses c
9      ON s.student_id = c.student_id
10     AND c.course_name = 'Math'; -- Math 课程 NULL
11
12 -- ✅ 数据保留 → IS NULL 过滤 WHERE
13 SELECT * FROM students s
14 LEFT JOIN courses c ON s.student_id = c.student_id
15 WHERE c.student_id IS NULL;
```

ON = JOIN 过滤 WHERE = JOIN 数据保留

PART 04





```
1  -- 0000000000000000
2  -- students ← enrollments → courses
3
4  SELECT
5      s.name AS student_name,
6      c.course_name,
7      e.enrollment_date
8  FROM students s
9  INNER JOIN enrollments e ON s.student_id = e.student_id
10 INNER JOIN courses c ON e.course_id = c.course_id;
```

00000

UNIQUE KEY (student_id, course_id) 00000

 JOIN

 INNER JOIN 0000000000000000

NO GRAPH
NO GRAPH
NO GRAPH
NO GRAPH
NO GRAPH

JOIN □□□

NO GRAPH
NO GRAPH
1
NO GRAPH
NO GRAPH
NO GRAPH
ON

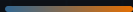
```
1  -- ✖ 遅い
2  SELECT * FROM students s
3  INNER JOIN courses c;
4  -- 遅い = 遅い × 遅い
```

NO GRAPH
NO GRAPH
2
NO GRAPH
NO GRAPH
NO GRAPH
NO GRAPH
NO GRAPH
NO GRAPH
NO GRAPH

```
1  -- ✔ 速い
2  CREATE INDEX idx_courses_student_id
3  ON courses(student_id);
```

DEBUG FLOW

1. EXPLAIN 実行 → 2. ON 実行 → 3. 実行時間 → 4. JOIN 実行 5. 実行時間

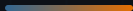


DO

- ✓ `students AS s`
- ✓ `SELECT *`
- ✓ `LEFT JOIN` `RIGHT JOIN`

DON'T

- × `ON` →
- × `LEFT` `INNER` →
- × `WHERE` → `LEFT` `INNER`



INNER = □□

□□□□□□□□

LEFT = □□

NOT NULL
NOT NULL
NOT NULL + IS NULL □□□□□

ON vs WHERE

JOIN NOT NULL
NOT NULL
NOT NULL
NOT NULL vs JOIN □□□

□□

NOT NULL
NOT NULL
NOT NULL
NOT NULL
NOT NULL
NOT NULL
NOT NULL ON □□

□□□□□□□□□□□□□□□□ — □ = LEFT□□□ = INNER□

Tables joined.

Table 1 JOIN Table 2

```
$ EXPLAIN SELECT ... FROM a LEFT JOIN b ON ...  
inner → left → on/where → index
```